



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 3

Team No.: 6

Project Title: Database Client-Server Management System

Team Members

Roll Number	Student Name	Signature
2520030001	M. Aswitha Patel	Aswitha
2520030414	N. Yishnaani	Yishnaani

1. Abstract

This project proposes a Linux-based database client-server system in C for managing a centralized university student database containing student information, marks, attendance, and course records. The system is designed to handle multiple clients accessing shared data simultaneously while preventing data corruption and maintaining consistency.

The project applies key Operating Systems and System Programming concepts. The server uses Linux system calls and kernel services for system and file operations. Multiple client requests are handled using processes and POSIX threads, with worker threads processing requests concurrently. A shared request and response queue is used for communication, while signals and POSIX signal handlers support configuration reload and controlled server shutdown.

Mutexes, condition variables, and counting semaphores are used to coordinate threads, protect shared database structures, and control access to shared resources. The system supports insert, search, update, and delete operations. Student records are managed using file abstractions, file descriptors, and Linux file I/O system calls. Memory-mapped file I/O is used for efficient access to frequently used data. Transaction and server activity logs are maintained, and active client connections and server resources are monitored.

The expected outcome is a reliable and efficient concurrent database server demonstrating practical applications of process control, IPC, signals, memory management, file I/O, POSIX threads, and synchronization.

2. Problem Statement

A university maintains a centralized database containing important student information such as personal details, marks, attendance, and course records. Multiple faculty members and administrative users may access and modify this shared data at the same time. Without proper coordination, simultaneous database operations can result in race conditions, inconsistent data, incorrect updates, and corruption of shared resources.

The project aims to develop a Linux-based database client-server system in C that can safely handle requests from multiple clients concurrently. The system must provide operations such as insert, search, update, and delete while ensuring that shared database data is accessed in a controlled manner.

From an Operating Systems and Systems Programming perspective, the problem involves managing concurrent processes and threads, communication between clients and the server, synchronization of shared resources, memory-mapped file access, file I/O, and signal handling. Appropriate POSIX threads, mutexes, condition variables, semaphores, signals, and Linux system calls are therefore required to coordinate concurrent activities and maintain reliable operation.

The project addresses the challenge of designing a system that provides efficient concurrent access while maintaining data consistency, synchronization, and controlled resource usage in a Linux environment.

3. Objectives

1. To develop a Linux-based client-server database system in C capable of accepting and processing requests from multiple clients concurrently.
2. To implement POSIX threads, mutexes, condition variables, and counting semaphores to coordinate concurrent requests, protect shared database structures, and prevent race conditions.
3. To implement database operations such as insert, search, update, and delete using Linux file I/O and memory-mapped file I/O, along with transaction and server activity logging.
4. To implement signals and POSIX signal handlers for configuration reload and controlled shutdown, while monitoring active client connections and server resources for reliable server operation.

4. Proposed Methodology

The project will be implemented in Linux using C with a client-server architecture. The server will accept requests from multiple clients and place them in a shared request queue. POSIX worker threads will process requests concurrently, while mutexes, condition variables, and semaphores will synchronize access to shared data and resources. The system will support insert, search, update, and delete operations using Linux file I/O and memory-mapped file I/O for frequently accessed data. Signals will handle configuration reload and controlled shutdown. Transaction and server activity logs will record operations, while active client connections and server resources will be monitored.

Project Flow:

Client → Server → Request Queue → Worker Thread → Database Operation → Response → Client

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / SystemCall	Purpose in the Project
POSIX Threads	Creates worker threads to process client requests concurrently.
Mutexes	Protect shared database structures from race conditions.
Condition Variables	Coordinates worker threads and request processing.
Counting Semaphores	Controls concurrent access to shared resources.
Signals & POSIX Signal Handlers	Handles configuration reload and controlled shutdown.
Process Management	Manages server and client processes during execution.
IPC	Supports communication between the client and server.
File Descriptors	Provides access to database and log files.
Linux File I/O System Calls	Performs reading, writing, and updating of database records and logs.
Memory-Mapped File I/O (mmap)	Provides efficient access to frequently used database data.
Virtual Memory	Supports the memory abstraction used by the memory-mapped database data.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030414	Yishnaani	Client & Server Concurrency: Implement client-server communication, request and response queues, server-side request handling, POSIX worker threads, and coordination of concurrent client requests using mutexes, condition variables, and counting semaphores.
2520030001	Aswitha Patel	Database & System Management: Implement insert, search, update, and delete operations using Linux file I/O and file descriptors, memory-mapped file I/O using mmap(), transaction and server activity logs, POSIX signal handling for configuration reload and controlled shutdown, and monitoring of active clients and server resources.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating system environment for developing and running the client-server system.
C / C++	Used to implement the client, server, database operations, threads, synchronization, signals, and file handling.
GCC	Compiles the C source code and builds the client-server programs.
POSIX Threads (Pthreads)	Creates and manages worker threads for concurrent request processing.

Linux System Calls / APIs	Used for file I/O, memory mapping, signals, and other system-level operations.
---------------------------	--

8. Expected Outcome

The system is expected to allow multiple clients to connect to the Linux server simultaneously and perform student database operations such as insert, search, update, and delete. Client requests will be placed in a shared request queue and processed concurrently by worker threads. Synchronization mechanisms will ensure safe access to shared data and prevent race conditions.

The system will generate appropriate responses for each client request, along with transaction and server activity logs showing database operations and server activities. Memory-mapped file I/O will provide efficient access to frequently used data. Signals will support configuration reload and controlled server shutdown.

The expected outcome is a reliable and efficient concurrent database client-server system that maintains data consistency while handling multiple requests. The project will demonstrate the practical use of POSIX threads, IPC, synchronization, signals, memory management, and Linux file I/O in a real-world student database application.

9. GitHub Repository

GitHub Repository Link: https://github.com/aswithapatel-0518/Database-Client-Server-Management-System-OSSP_PROJECT

Repository Created: Yes

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: K. Hema

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____